UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

UML INTRODUCTION

Francesco Guerra
francesco.guerra@unimore.it

Models

- Systems and Software Systems
  - System: an integrated whole composed of components that interact and influence each other, perceived as a single unit with a specific task or purpose.
  - In Information Technology, we focus especially on software systems, i.e., systems described and implemented through software models.
- Abstractions
  - Abstractions are simplified representations of reality that software systems are built upon.
  - Abstraction = Generalization: setting aside specific or individual features.
  - Abstracting = Process:
    - Moving away from details.
    - Distinguishing substance from incidental aspects.
    - Identifying common characteristics.

Models

- Characteristics of Models (Stachowiak, 1973)
- Mapping
  - A model is always a representation of something:
    - It maps natural or artificial originals.
- Reduction
  - A model does not reproduce the original in full detail.
  - It captures only the attributes relevant to the modeler or user: irrelevant aspects are deliberately omitted.
- Pragmatism
  - Models are oriented toward usefulness:
    - For whom? (the intended audience), Why? (the purpose), What for? (the task or problem being addressed)
  - A model is used instead of the original, but only within a specific context, timeframe, and purpose.

Quality of models [Bran Selic]

- **Abstraction**
  - A model is a **reduced representation** of the system it describes.
  - Irrelevant details are hidden or removed to make the **essence of the system easier to understand**.
- **Understandability**
  - Elements should be presented **intuitively**, e.g., using graphical notation.
  - Understandability derives from the **expressiveness,** i.e., the ability to represent complex content with **few concepts**, minimizing cognitive load.
- **Accuracy**
  - The model must highlight the **relevant properties** of the real system.
  - It should reflect reality **as closely as possible**.
- **Predictiveness**
  - The model should allow **predictions of interesting, non-obvious properties** of the system.
  - Achieved via **simulation** or **formal analysis**.
- **Cost-effectiveness**
  - Over the long term, it must be **cheaper to create and use the model** than to directly create or experiment with the system itself.

What Is the UML?

- UML is a family of graphical notations designed to help describe and design software systems, particularly those built using the object-oriented (OO) style.
  - Programming languages alone are often too low-level to discuss or reason effectively about software design.
  - Graphical modeling languages allow teams to communicate design decisions clearly and reason at a higher level of abstraction.
  - There has been significant debate about actual role and utility of graphical modeling languages.
- UML is a relatively open standard, maintained by the Object Management Group (OMG).
  - The OMG was created to develop standards that ensure interoperability across tools, platforms, and organizations.

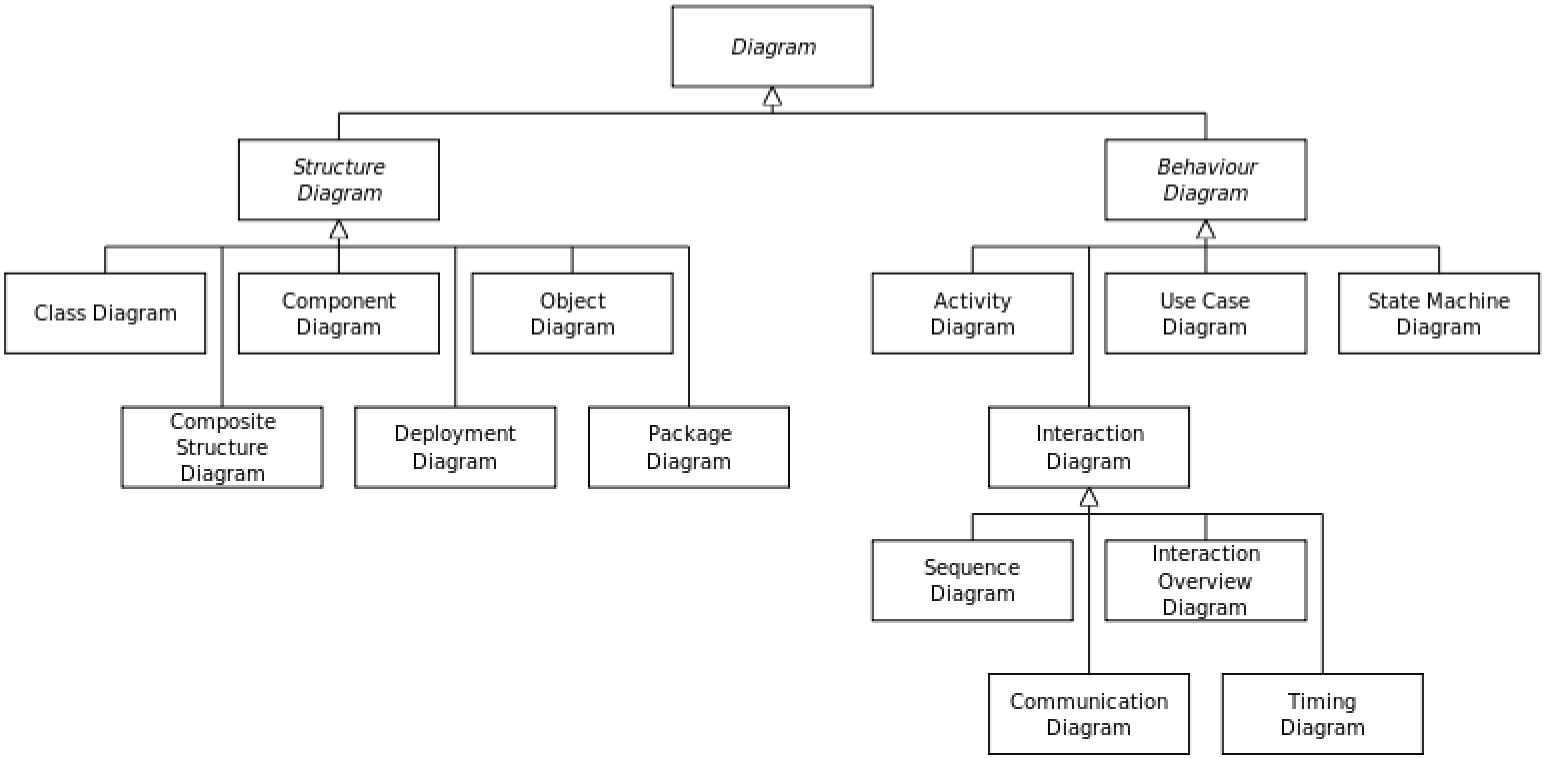
UML timeline

- UML 1.x (1997–2001)
  - 1997: UML 1.0 adopted as OMG standard.
  - Introduces main diagrams: class, use case, state, sequence, activity.
- UML 2.0 (2005)
  - Major revision with enhanced semantics and rigorous metamodel architecture (MOF).
  - Improved behavioral diagrams: activity diagrams redesigned, sequence diagrams more detailed.
  - Added interaction and communication diagrams.
  - Promoted use in Model Driven Architecture (MDA).
- UML 2.5.x (2015–2017)
  - Simplification and consolidation of UML specification.
  - UML 2.5.1 (2017) is the latest stable revision.
  - Industrial interest declines; lighter modeling notations like C4, BPMN, DSL are more common.

Ways of Using the UML

- Three modes in which people use the UML:
  - Sketch
    - to help communicate some aspects of a system
    - Forward and reverse engineer
    - Their emphasis is on selective communication rather than complete specification.
  - Blueprint
    - Completeness
    - Designers develop blueprint models and developers the details of implementation
    - Blueprints require much more sophisticated tools than sketches do in order to handle the details required for the task.
  - Programming language
    - The developers draw UML diagrams that are compiled to executable code

UML Diagrams



What Is Legal UML?

- Legal UML: UML elements and diagrams that are well-formed according to the specification.
- Descriptive vs Prescriptive Rules
  - Prescriptive rules:
    - Defined by an official body.
    - Specify what is legal or illegal and what meaning should be assigned to statements in the language.
  - Descriptive rules:
    - Describe how the language is actually used, not how it should be used.
    - UML is mainly descriptive, similar to blueprints in traditional engineering.
- Implications for UML Diagrams
  - Diagrams may contain information relevant only for that particular diagram.
  - Absence of an element does not imply anything: you cannot infer properties of the system from what is missing.
  - UML is precise, but it does not enforce prescriptive semantics; it documents intent, not obligations.

The Meaning of UML

- The specification doesn't say about what the UML means outside of the UML meta-model.
- No formal definition exists of how the UML maps to any particular programming language.
  - You cannot look at a UML diagram and say exactly what the equivalent code would look like.
  - You can get a rough idea of what the code would look like.
- UML Is Not Enough!
  - Different diagrams can be useful, and you shouldn't hesitate to use a non-UML diagram if no UML diagram suits your purpose.

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

References

- Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.